UNIVERSITY
OF WOLLONGONG
IN DUBAI

Assignment Cover Sheet

Subject Code:	ECTE331		
Submission Type:	Project		
Assignment Title:	Part 2		
Student Name:	Shannen Godinho	Student Number:	8238182

Introduction

This report presents the design, implementation, and validation of a synchronized multi-threaded Java application designed to coordinate two cooperating threads with strict execution dependencies. To avoid race conditions and assure proper changes to shared variables, we use resource-efficient synchronization algorithms that enforce the appropriate execution order without using CPU-intensive active waiting or sleep approaches. Finally, we validate the system's correctness using both mathematical verification of the shared variable states and empirical stress testing across many repetitions.

a) Final Correct Values of the Shared Variables

- Calculate (FuncA1) $A_1 = \sum_{i=0}^{500} i = \frac{500 \times 501}{2} = 250 \times 501 = 125{,}250$
- Calculate (FuncB1) $B_1 = \sum_{i=0}^{250} i = \frac{250 \times 251}{2} = 125 \times 251 = 31{,}375$
- Calculate (FuncB2) $B_2 = A_1 + \sum_{i=0}^{200} I\ B_2 = 125{,}250 + \frac{200 \times 201}{2} = 145{,}350$
- Calculate (FuncA2) $A_2 = B_2 + \sum_{i=0}^{300} i\ A_2 = 145{,}350 + \frac{300 \times 301}{2} = 190{,}500$
- Calculate (FuncB3) $B_3 = A_2 + \sum_{i=0}^{400} i\ B_3 = 190{,}500 + \frac{400 \times 401}{2} = 270{,}700$
- Calculate (FuncA3) $A_3 = B_3 + \sum_{i=0}^{400} i\ A_3 = 270{,}700 + 80{,}200 = 350{,}900$

b) Thread Synchronisation Design

To enforce execution order without busy-waiting or sleeping, we use four binary semaphores, all initialized with 0 permits:

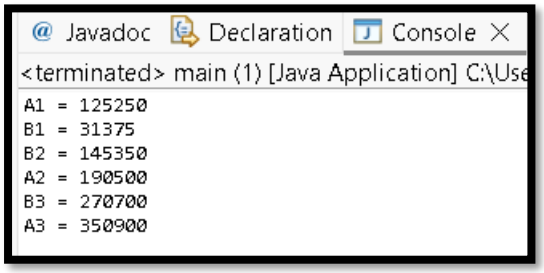
1. a1Done: Signaled by Thread A when FuncA1 finishes. Thread B waits on it before starting FuncB2.
2. b2Done: Signaled by Thread B when FuncB2 finishes. Thread A waits on it before starting FuncA2.
3. a2Done: Signaled by Thread A when FuncA2 finishes. Thread B waits on it before starting FuncB3.
4. b3Done: Signaled by Thread B when FuncB3 finishes. Thread A waits on it before starting FuncA3.

Synchronization Execution Flow: Thread A and Thread B perform FuncA1 and FuncB1 simultaneously, as neither function requires any external precondition. However, Thread B cannot go to FuncB2 until it receives a1Done, putting it in a blocked state if Thread A is still running. When Thread A finishes FuncA1 and writes to the shared variable A1, it calls a1Done.release(), which instantly frees Thread B to begin FuncB2. Meanwhile, Thread A's execution is halted since it is blocked on b2Done.acquire(), awaiting the completion of FuncB2.

When Thread B completes FuncB2 and changes B2, it tells Thread A by using b2Done.release(), which allows Thread A to perform FuncA2. This lockstep continues as Thread B waits on a2Done.acquire() until Thread A completes FuncA2 and signals with a2Done.release(), allowing Thread B to compute FuncB3. Finally, Thread A stalls on b3Done.acquire() until Thread B completes FuncB3 and signals with b3Done.release(), allowing Thread A to run its final function, FuncA3. This sequence prevents active waiting and guarantees that all cross-thread dependencies are fully enforced.

c) Code uses the precise semaphore synchronization mechanism outlined in Part (b). To conduct the summation procedures, a special utility class entitled LoopSumCal was created, which has a static function calculateSum() that uses a while loop instead of a closed-form formula. The thread execution logic is written within a class named main, which initializes and executes Thread A and Thread B, ensuring that all cross-thread arithmetic computations are securely coordinated.

OUTPUT



d) To test the synchronization model's accuracy and safety, the main() function was modified to run the concurrent thread sequence over 5,000 times. Thread scheduling is non-deterministic, hence race situations and deadlocks may not occur during a single execution. Running a large number of repetitions stresses the code under different CPU demands. At the beginning of each run, the shared state variables are reset and the coordination semaphores are re-initialized to 0.The main thread waits for both threads to terminate using the join() function before asserting the calculated values against the mathematically accurate values

from Part (a). All 5,000 iterations reliably match the predicted computations, demonstrating that the implementation is resilient, predictable, and devoid of race situations or deadlocks.

OUTPUT